MASTER PROMPT FOR CLAUDE — SLOTSYNC

Complete, phased Antigravity build system for a production-quality hackathon MVP

Deadline: October 29, 2026, at 11:00 AM IST

Primary goals: exceptional UI/UX, functional end-to-end workflows, Cloudflare-first architecture, reliable reminders, honest Instinct usage, and no missing blueprint requirements.

Use this document as the complete master prompt. Upload the latest SlotSync blueprint PDF and the Instinct reminder example/README to Claude alongside it.

Full prompt begins on the next page.

MASTER PROMPT FOR CLAUDE — SLOTSYNC

Generate a Complete, Phased Antigravity Build System for a Production-Quality Hackathon MVP

Act as an elite product architect, senior full-stack engineer, UI/UX design director, Cloudflare specialist, QA lead, and experienced hackathon judge.

Your task is to create a complete sequence of highly detailed, copy-paste-ready prompts for Google Antigravity that will build SlotSync into a polished, fully functional, end-to-end application.

Do not implement the application yourself. Your primary deliverable is the phased prompts that I will execute in Antigravity.

I have very limited development time. My deadline is October 29, 2026, at 11:00 AM IST. I need a reliable, demo-ready project with exceptional visual quality and complete core workflows. I cannot afford careless omissions, contradictory instructions, unnecessary rewrites, broken features, or a frontend full of mock data.

1. SOURCE-OF-TRUTH REQUIREMENTS

I will provide you with:

1. The latest SlotSync project blueprint PDF.

2. The Instinct reminder example and its README, if available.

3. Potentially, an existing repository or screenshots.

Read the entire available blueprint and supporting files before creating your phases.

Rules:

- Treat the latest blueprint as the primary product requirements source.
- Extract every feature, screen, workflow, constraint, and acceptance criterion.
- Preserve the intended product and terminology.
- Do not silently omit, simplify away, or replace a requirement.
- Do not invent a feature and claim it was specified in the blueprint.
- If the blueprint has ambiguity or conflicting requirements, identify it and recommend the safest practical interpretation.
- Treat the Instinct sample as reference material, not production-ready code.
- If repository files are available, inspect the actual code before recommending changes.
- Clearly distinguish required features, recommended improvements, and optional stretch goals.
- Do not ask me to repeat information already present in the provided files.

Before writing any build prompt, produce a requirements traceability matrix containing:

- Requirement ID
- Exact requirement or faithful summary
- Source document/section
- User value
- Required screen or workflow

- Backend/data requirements
- Acceptance test
- Planned phase
- Priority: P0, P1, or P2
- Status

Use this matrix throughout your planning to prevent missing features.

2. PRODUCT VISION

Product name: SlotSync

Core problem:

Local service providers manage appointments through WhatsApp messages, calls, and walk-ins. This creates double bookings, missed appointments, poor visibility, and repetitive manual work.

SlotSync should let a service provider manage appointments in one place while giving customers a fast, frictionless way to book without installing an app.

The product must feel like a real commercial SaaS product that could be used by a salon, tutor, repair technician, consultant, clinic, or other appointment-based small business.

The central promise is:

“One simple booking link for customers. One reliable appointment workspace for the business.”

Do not turn this into a generic AI chatbot or an overcomplicated enterprise system.

3. NON-NEGOTIABLE PRIORITIES

Priority A — Exceptional UI/UX

Visual design is one of the most important judging criteria.

I do not want a generic AI-generated dashboard with basic cards, random gradients, inconsistent spacing, or default component styling.

Create a cohesive, premium, commercially credible design system.

The interface should have:

- Excellent visual hierarchy.
- Carefully selected typography.
- Consistent spacing and layout rules.
- Refined colors and contrast.
- Elegant but restrained shadows and borders.
- Beautiful empty, loading, success, and error states.
- Thoughtful micro-interactions.
- Responsive layouts for desktop, tablet, and mobile.
- Accessible forms, keyboard navigation, visible focus states, and readable text.

- Clear navigation and consistent component behavior.
- Well-designed charts only where they communicate useful information.
- Realistic, restrained sample content where needed for a clearly labelled demo.
- No visual clutter or unnecessary animations.

Choose one strong design direction and maintain it across the entire product. Explain the design system briefly before implementation.

The public booking experience must be especially polished and mobile-first. A customer should understand how to book with minimal friction.

The owner dashboard must feel professional and information-dense without becoming overwhelming.

Priority B — Functional completeness

Every visible primary action must work.

No:

- Dead buttons.
- Placeholder pages disguised as completed features.
- Fake success messages.
- Hardcoded availability presented as live.
- Fake charts or fabricated analytics.
- Forms that do not persist data.
- UI controls that do not affect actual state.
- Broken navigation.
- Separate demo databases disconnected from the real application.
- Critical workflows implemented only in frontend state.

Use actual backend operations for important actions. Show clear errors when an operation fails.

Priority C — Cloudflare-first architecture

Use Cloudflare as the primary hosting and backend platform.

Prefer a simple, coherent architecture using:

- React and TypeScript for the frontend.
- Cloudflare Workers for backend APIs.
- Cloudflare D1 for relational application data.
- Cloudflare Cron Triggers for scheduled reminder processing.
- Cloudflare Queues only if they materially improve reliability without risking the deadline.
- Cloudflare R2 only if file storage is genuinely needed.
- Resend for email delivery, subject to sender/domain verification requirements.

Use Cloudflare Pages or Workers static assets according to the current project's architecture and deployment needs. Do not create two competing deployment approaches.

Verify current Cloudflare APIs and configuration against official documentation when necessary. Do not rely on outdated package APIs.

Avoid unnecessary infrastructure, excessive abstractions, and dependencies that cannot be configured before the deadline.

Priority D — Instinct usage

The organiser said it would be good to use Instinct.

Use Instinct meaningfully, but do not invent integration capabilities.

The Instinct response I received stated that the current Instinct WhatsApp chat does not have a customer-messaging gateway, a Cloudflare Workers/D1 connection, or an inbound webhook endpoint for SlotSync.

Therefore:

- Use Instinct as a genuine development and workflow assistant where appropriate.
- It can help review the implementation, debug failures, improve the reminder workflow, and prepare the demo.
- Keep appointment data, scheduling logic, access control, and customer messaging in SlotSync's actual application architecture.
- Do not claim that Instinct sends customer notifications unless a real, verified integration exists.
- Do not fabricate an Instinct API, webhook, SDK, or connection.
- Do not make Instinct a runtime dependency unless an officially supported integration is verified and the organiser's requirements justify it.
- Document the real contribution of Instinct for the final presentation.
- If direct product integration is expected, identify the missing integration information instead of pretending it exists.

The application must remain functional even if Instinct is unavailable.

4. REQUIRED PRODUCT CAPABILITIES

Use the blueprint as the final authority. The following capabilities are important and must be checked against it, not blindly substituted for it.

A. Public customer booking

Build a polished, shareable public booking page with:

- Business identity and branding.
- Service list, duration, and pricing.
- Available dates and time slots calculated from real business rules.
- Customer information form with validation.
- Booking confirmation and unique reference.
- Clear next steps after booking.
- Reschedule and cancellation flows when allowed by business policy.
- Proper handling of unavailable slots, expired sessions, and network failures.
- Mobile-first interactions and accessible forms.

Customers should not need to create an account or install an application unless the blueprint explicitly requires otherwise.

B. Business onboarding and settings

Include the required workflows for:

- Business registration and login.
- Business profile and branding.
- Service creation, editing, pricing, and duration.
- Working hours, breaks, holidays, and blocked periods.
- Staff or resource configuration if required by the blueprint.
- Booking policies and cancellation rules.
- Public booking link and sharing options.

C. Owner dashboard and calendar

Include:

- Useful daily overview.
- Upcoming appointments and status breakdowns.
- Calendar/list views as specified by the blueprint.
- Search, filters, and clear status indicators.
- Create appointment manually through a Quick Add flow.
- Support for appointments originating from WhatsApp, phone calls, walk-ins, and public booking links.
- Booking detail view with permitted status changes.
- Reschedule, cancel, and confirm operations.
- Clear empty and error states.

D. Customer management and history

Where required by the blueprint:

- Customer directory.
- Search and filtering.
- Customer profile.
- Previous and upcoming appointments.
- Appointment history and relevant notes.
- Safe duplicate handling and useful customer context.

E. Reminders and notifications

Implement the reminder workflow using the actual SlotSync backend and supported providers.

Requirements:

- Booking confirmation.
- Appointment reminders at the configured interval.
- Cancellation and rescheduling behavior.

- Accurate local time and timezone handling.
- Notification delivery status and visible failures.
- Retry behavior with sensible limits.
- Idempotency protection against duplicate sends.
- No reminders for cancelled appointments.
- Recalculation after rescheduling.
- Clear separation between notification creation and successful delivery.
- Safe handling of missing provider credentials.

Use Cloudflare Cron Triggers and Resend if they are compatible with the existing architecture and can be configured in time.

If WhatsApp delivery cannot be properly configured before the deadline, implement and test email first. Clearly label WhatsApp as unavailable or demo-only rather than claiming it works.

Review the provided Instinct reminder sample critically. In particular, assess:

- Whether its rolling 24-hour scan really sends reminders at the intended time.
- Duplicate-send risk.
- Cancellation or rescheduling during an in-flight send.
- Tenant isolation.
- Authentication of operational endpoints.
- Sender verification and provider restrictions.
- Timezone assumptions.
- Placeholder database IDs and secrets.

Do not copy known limitations into the final system without addressing them or explicitly documenting residual risks.

F. Reviews, distribution, and analytics

Implement the blueprint's required:

- Review and feedback flow.
- Public booking link sharing.
- QR code if specified.
- Business performance insights.
- Appointment and customer metrics.
- Privacy and account settings.

Metrics must come from actual stored application data. Never fabricate business performance figures.

5. DATA, SECURITY, AND RELIABILITY REQUIREMENTS

Design and verify the data model before building all screens.

Requirements:

- Proper relational schema, foreign keys, indexes, and migrations.
- Tenant isolation at the backend, not just through frontend filtering.

- Authentication and server-side authorization.
- Secure password/session handling using an appropriate, maintained approach.
- Input validation on all write operations.
- Protection against overlapping appointments and race conditions.
- Consistent appointment status transitions.
- Correct handling of local business timezones, daylight saving where relevant, and UTC storage.
- Safe rescheduling and cancellation.
- Idempotent notification processing.
- Bounded retries and clear failure reporting.
- No API keys or secrets in frontend code.
- No real customer data in demo fixtures.
- No public exposure of privileged operational endpoints.
- Useful logs without exposing sensitive customer details.
- Database constraints for critical invariants where possible.

Never trust the frontend to enforce booking availability, authorization, pricing, or business ownership.

Before implementation, choose the simplest reliable approach to booking concurrency on Cloudflare D1. Explain its limits and test it. Do not claim a race condition is solved unless the implementation and tests support that claim.

6. VISUAL DESIGN WORKFLOW — DO NOT SKIP

Before implementing the whole app, make Antigravity establish the visual direction.

Have it:

1. Inspect the existing repository and any supplied screenshots.

2. Define the design system and reusable components.

3. Build the highest-value visual foundation first.

4. Implement the public booking experience and the owner dashboard as real, working vertical slices.

5. Verify responsive behavior before copying the patterns across all remaining pages.

6. Maintain consistency in spacing, typography, component sizing, forms, dialogs, tables, navigation, and feedback states.

The design must be attractive because of strong product design, not because every screen contains gradients or animations.

Create reusable components for:

- Buttons and action states.
- Form controls and validation.

- Cards and metric summaries.
- Status badges.
- Navigation and page headers.
- Dialogs and confirmation prompts.
- Toasts and inline feedback.
- Loading skeletons.
- Empty states.
- Error and retry states.
- Calendar and appointment items.
- Responsive mobile navigation.

Do not use a component library blindly. Customize components to match the chosen visual identity.

If screenshots or visual previews can be generated in Antigravity, review them and correct layout problems before moving on.

7. BUILD STRATEGY: PHASED PROMPTS

Produce a complete sequence of numbered Antigravity prompts. Each phase must be independently copy-pasteable and must build on the previous phase.

Use these phases as a starting structure, adjusting them when the blueprint requires it.

Phase 0 — Repository inspection and requirement lock

- Inspect the existing repository, package manager, scripts, schema, routes, tests, and deployment setup.
- Read the blueprint and reminder files.
- Produce the traceability matrix.
- Identify existing code that must be preserved.
- Establish deadline-aware scope and a build order.
- Do not rewrite the repository unnecessarily.

Phase 1 — Architecture and data foundation

- Finalize the architecture, data model, migrations, validation, auth strategy, and API contracts.
- Identify all required environment variables and secrets.
- Resolve critical ambiguities before implementation.

Phase 2 — Design system and premium UI foundation

- Establish the visual direction.
- Build reusable responsive components and layout.
- Implement the visual foundation without creating fake completed functionality.

Phase 3 — Authentication, business onboarding, and configuration

- Implement real authentication, business profile, services, hours, and policies.

Phase 4 — Public booking vertical slice

- Implement real availability, slot selection, booking creation, confirmation, and customer-facing error states.

Phase 5 — Owner dashboard and appointment management

- Implement calendar/list views, Quick Add, appointment details, status transitions, cancellations, and rescheduling.

Phase 6 — Customer directory and history

- Implement customer records and appointment history.

Phase 7 — Reminders, notification settings, and Instinct-assisted workflow

- Integrate supported email delivery and scheduled reminders.
- Review and correct the sample's known limitations.
- Keep Instinct's role truthful and separate from runtime integrations unless verified.

Phase 8 — Reviews, sharing, QR, analytics, and remaining blueprint features

- Implement every remaining required feature.
- No feature may be dropped simply because it is less visually prominent.

Phase 9 — Integration, security, and regression testing

- Verify cross-feature behavior and remove inconsistencies.
- Run available automated tests and fix failures.

Phase 10 — Deployment and production-like verification

- Deploy using the selected Cloudflare architecture.
- Verify migrations, secrets, public booking, owner workflows, and notification configuration.
- Report actual deployment status and remaining limitations.

Phase 11 — Final polish, demo script, and submission readiness

- Check the application on mobile and desktop.
- Test every primary action.
- Prepare a truthful hackathon demo flow.
- Create a short explanation of the problem, solution, architecture, Instinct usage, and limitations.

Do not force a phase into the plan if it duplicates existing code. Adapt the prompts to the actual repository.

8. EACH PHASE PROMPT MUST HAVE THIS STRUCTURE

For every phase, provide:

1. Phase name and objective.

2. Dependencies and prerequisites.

3. Exact copy-paste Antigravity prompt.

4. Files and areas to inspect first.

5. Implementation requirements.

6. UI/UX requirements, including relevant screen behavior.

7. Backend and database requirements.

8. Error handling and edge cases.

9. Security requirements.

10. Tests to create or update.

11. Acceptance criteria.

12. Manual verification checklist.

13. Definition of done.

14. A concise completion report Antigravity must return.

15. Explicit instruction not to move on while critical tests or requirements fail.

The prompt must direct Antigravity to work in the actual repository, inspect before modifying, preserve working features, make minimal targeted changes, and report the actual result.

Every phase prompt must be detailed enough that Antigravity does not need to guess what to build.

Do not use phrases such as “implement the remaining features” without listing those features.

Do not use vague instructions such as “make it beautiful” without explaining the expected UX and design quality.

9. DEADLINE-AWARE IMPLEMENTATION

The deadline is tomorrow, October 29, 2026, at 11:00 AM IST.

Optimize for a complete, stable, visually impressive demo, not unnecessary technical complexity.

Require Antigravity to:

- Inspect first and reuse working code.
- Build complete vertical slices.
- Prioritize P0 requirements and end-to-end user flows.
- Use the existing package manager and project conventions.

- Avoid unnecessary dependencies and rewrites.
- Keep each phase small enough to verify.
- Run focused tests after meaningful changes.
- Run full tests after integration.
- Keep a running checklist of completed and uncompleted requirements.
- Reserve meaningful time for final testing and the live demo.
- Stop adding optional features when the remaining time threatens stability.

However, do not silently drop mandatory blueprint features. If a feature cannot be completed, report the exact gap and the safest fallback.

Prioritize the actual time available rather than inventing an unrealistic timeline. Identify which features must be built first and which optional refinements can wait.

10. QUALITY GATES

Every phase must pass these checks before it is considered complete.

Product:

- The intended workflow works from start to finish.
- Data persists in the actual database.
- Failure states are handled.
- Permissions are enforced server-side.

UI/UX:

- Desktop and mobile layouts are usable.
- The visual design is consistent.
- Forms provide useful validation and feedback.
- No dead primary buttons or broken navigation.
- Loading, empty, success, and error states are implemented where relevant.

Engineering:

- Existing scripts and project conventions are preserved.
- Relevant tests pass.
- No hardcoded secrets.
- No obvious tenant-isolation vulnerabilities.
- No unrelated features are broken.
- Migrations are repeatable and documented.

Honesty:

- Never report a test as passed unless it was actually run and passed.
- Never report deployment as complete unless it was actually deployed and verified.
- Never report email or WhatsApp delivery as successful based only on mocked requests.
- Distinguish implementation, local testing, provider testing, and deployed verification.
- Record any remaining blockers explicitly.

11. FINAL AUDIT DELIVERABLES

After generating all phases, also provide:

1. The full requirements traceability matrix.

2. A dependency map between phases.

3. A master checklist of all screens, features, APIs, database entities, and tests.

4. A list of required environment variables and where each belongs.

5. A Cloudflare deployment checklist.

6. A reminder-provider configuration checklist.

7. A risk register covering the deadline, authentication, booking conflicts, timezone handling, notification delivery, and deployment.

8. A final acceptance-test checklist.

9. A concise live-demo script that demonstrates the problem and the complete customer-to-owner workflow.

10. A truthful explanation of how Instinct was used.

11. A list of features that must never be represented as working if they are still mocked or unconfigured.

12. A recovery procedure if a phase breaks a previously working feature.

12. OUTPUT RULES

- First, read the supplied sources and report the product requirements you extracted.
- Then provide the traceability matrix and phased build prompts.
- Make every phase prompt independently copy-pasteable.
- Put each full Antigravity prompt in its own clearly labelled code block or separate section.
- Do not compress later phases into vague summaries.
- Do not start implementing the application.
- Do not claim the application is already built.
- Do not generate speculative credentials, URLs, API keys, or database IDs.
- Do not invent Instinct integration capabilities.
- Do not assume a provider is configured just because code exists.
- Do not remove any blueprint requirement without explicitly identifying it.
- Use sensible defaults where the source is clear, rather than delaying the plan with unnecessary questions.

- If context or output limits prevent you from generating every detailed phase in one response, stop at a complete phase boundary. State exactly which phase comes next and continue when I ask. Never truncate a prompt halfway through.
- Ensure all phases are consistent with one architecture, one data model, one design system, and one set of requirements.

Your objective is to give me a complete, reliable execution playbook for building SlotSync in Antigravity before the deadline, with outstanding UI/UX and no preventable omissions.